Document number: P3265R3
Audience: EWG

Ville Voutilainen
2024-05-28

Ship Contracts in a TS

Abstract

This paper suggests that Contracts should ship in a Technical Specification first, not in the C++26 IS.

The motivation for this is many-fold. We lack

- implementation experience
- more importantly, field experience in the form of deployment and usage experience
- WG21-wide design consensus on multiple points in P2900
- cohesion and coverage; we don't have WG21-wide consensus on how to deal with virtual functions, how to deal with coroutines, and how to deal with pointers to functions
- increased safety; contract assertions are subject to as much undefined behavior as the rest of the language is, and this may well be very problematic both for the runtime checks written in contract assertions and also for the chances of static analysis tools' ability to correctness-check code with the help of contract assertions.

Additional ruminations

There isn't actually much to add.

For those unaware, we have had language TSes before, for Concepts, Modules, and Coroutines. Those were, regardless of the grumbling about the time and effort they took, all successful in the sense that we got better language features due to using a TS, with fewer if any unaddressed concerns, with a better understanding, and better consensus in WG21.

We have various design disagreements in SG21, and based on EWG's look at Contracts in Tokyo, fair amounts of disagreements between what SG21 is currently proposing and what some portions of EWG think about it. We can certainly try to resolve all such differences and ship the result in C++26, but that seems unwise. We should hopefully have more than paper-exercise disagreements that are resolved on paper - we should try to resolve those disagreements with something more concrete than papers in our hands, and that something more concrete should be implementations we can play with.

And that includes implementations of different alternatives instead of just one, to have an antithesis for the thesis, with the goal of producing a better synthesis.

Finally, there's been various discussions in SG21, including significant parts of the discussion on P2680 where statements along the lines of "if we want to ship this in C++26, we should do $foo and $bar, but not $baz" have been made. That is, the schedule goal for shipping the Contracts MVP has a significant impact on what we look at and when. While that's understandable for an MVP approach, it also runs into the danger of closing alternative design doors prematurely, when they don't fit into the MVP schedule and the timeline to make it into C++26 - even when they might be better doors to walk through.

So, the main goal of this idea, in addition to getting a better understanding, more field experience, and all that jazz, is to remove that urgency, and hopefully remove its significant impact on the design choices that we make. Decouple Contracts from such an immediate deadline, have a breather, gather more experience. Use the ISO trial balloon that is a Technical Specification to do it.

What questions should the TS answer?

Here's a start:

- is it ergonomic:
  - does constification cause issues with teaching? When reading? Does it catch more bugs than it causes? How cumbersome is it?
  - is it useful for its intended purpose or does it end up being used for something else?
  - can compilers provide a user-friendly interface to the implementation-defined features that users can integrate with their toolchains and actively use as intended by the design?
  - can users understand multiple evaluation?
- is it fully-featured?
  - are there immediate missing pieces one reaches for but they don't exist? for precedent, see lambdas in unevaluated contexts.
  - how badly do we need Lisa's full interfaces as opposed to just post-conditions with captures sugar?
  - how badly do we need labels?
- is it implementable?
  - is the proposal implementable in general, are there any surprises? Anything that needs to be fed back as a design consideration?
  - is the evaluation elision between postconditions/preconditions implementable in a useful way? (We might not get this from a compiler yet, it requires work, but I don't want to standardize something like `throw;` that prevents efficient exception implementations without thread-local storage. It might be we find that we have to allow more things than just side-effects to be ignored, like longjump.)

I'll add a couple more:

- Does it provide a base that we can build useful static analysis on?
- If we were to put both P2900 and P3285 (Gaby's proposal defaulting to 'strict' contracts, providing 'relaxed' contracts as an alternative) into the same TS, which approach is more viable and more useful?

DG's questions from P4000

- Is there an implementation?
  - There are bits of an implementation in early stages, the full proposal is nowhere near implemented. An additional concern about this is that we're nearing the design deadline for C++26, which gives us between little and no time to change the design based on implementation experience feedback or deployment experience feedback.
- Is it a Library or Language proposal, or both?
  - It is just a Language proposal.
- Is the proposal a foundational proposal, meaning many other C++ aspects/proposals depend on it, and/or does it depend on many other C++ aspects/proposals?
  - The proposal may end up having interactions with safety profiles.
- Is it independent on aspects of the language?
  - I'm not sure what this question means. As mentioned, there may be interactions with profiles.
- Are there competing design proposals?
  - Yes. P3285 is a competing approach, despite its formulation as a change proposal to P2900.
- Is the proposal so complicated or large to fear there will be error in design decisions?

    - Yes. All of the proposal's design has been done without implementation and deployment experience. It's a complete paper exercise, without prototyping cycles that could've informed the design and decisions thereof.
- Is it a research idea?
    - To some extent, kind of, parts of it are.
- Is there substantial invention?
    - Yes. Various parts of the design are inventive.
- Can it be staged?
    - I'm not sure what this question means. Splitting the proposal into artificially smaller parts doesn't seem to help. Postponing its adoption to the C++29 would help, but there are arguments in favor of a TS along the lines that a TS has WG21 approval, whereas any particular proposal still in the pipeline waiting for plenary approval does not, and can be changed in unexpected ways.
- Is there a subpart that deserves to be in IS?
    - Not that I can see. The concerns and approach disagreements impact the most basic parts of the proposed facility.
- Is the wording complicated or unconventional?
    - No.
- Is applying to library important for the feature?
    - There's disagreements on that point, but applying it to the standard library *specification* isn't necessary at this point. Getting implementation experience with applying the proposal to a standard library *implementation* would certainly be beneficial.
- Will the proposal benefit from early integration (can it be applied to a WP)?
    - No. There are concerns whether the proposed approach is the right one. Applying the proposal to a WP doesn't help with those concerns.
- Will you get feedback/testing only after TS publication or IS publication?
    - No, such feedback/testing can certainly happen earlier, but it will likely not happen early enough to provide feedback in time for C++26.
- Is there a motivation to slow down a proposal?
    - No. The motivation is to ship something that works, and is tested to work. If we have to slow things down to achieve that, so be it, but slowing down is not a motivation in and of itself.
- Explicitly state the acceptance criteria for the TS into IS.
    - That acceptance criteria is to have answers to the questions in the previous section of this paper.
- Are you juggling a large number of related or dependent proposals (other proposals that depend on this proposal)?
    - Not yet, but once the initial adoption is done, we are going to. And it would be rather nice if the initial proposal has been tested before adopting it and using it as a baseline for multiple further extensions.
- Are you aiming for user feedback?
    - Yes.
- Are you aiming for implementation feedback?
    - Yes.
- Is there a scheduling concern to make C++xx for it or its dependents? feedback?
    - Yes. As mentioned before, the current design is a paper exercise, and exploring certain alternatives has been rejected partially due to doubts about work on such alternatives approaches being feasible within the C++26 cycle.

Summa summarum

The proposal in this paper is to put both P2900 and P3285 into a TS, and invite implementation and deployment feedback, with the list of questions suggested earlier in this document.